

Laboratorio 10 — Aprendizaje Semi-Supervisado

CC3074 · Minería de Datos | Semestre I – 2026

Grupo #____

2026-05-17

Contents

1. Selección y Análisis del Dataset	1
1.1 Descripción del dataset	1
1.2 Dimensionalidad y estructura	2
1.3 Estadísticas descriptivas	2
1.4 Valores faltantes	3
1.5 Balance de clases	4
1.6 Distribución de variables	6
1.7 Boxplots por clase	8
1.8 Matriz de correlación	10
2. Preprocesamiento de Datos	12
2.1 Imputación de valores faltantes	12
2.2 Tratamiento de outliers (Winsorización)	12
2.3 Normalización Min-Max	13
2.4 Distribuciones antes vs. después del preprocesamiento	14
2.5 Resumen del preprocesamiento	16
3. Experimentación	16
3.1 Objetivo de la experimentación	16
3.2. Análisis de Resultados Experimentales	24
3.3 Sensibilidad a hiperparámetros	24
3.5 Sobreajuste y subajuste	26
3.6 Conclusión experimental	26

1. Selección y Análisis del Dataset

1.1 Descripción del dataset

El dataset utilizado es **Water Potability**, disponible públicamente en Kaggle:

<https://www.kaggle.com/datasets/adityakadiwal/water-potability>

Contiene mediciones fisicoquímicas de muestras de agua y una variable objetivo binaria que indica si el agua es apta para consumo humano (**Potability**: 1 = potable, 0 = no potable). El escenario semi-supervisado es realista: en la práctica, analizar y etiquetar muestras de agua requiere laboratorios especializados, por lo que contar con pocas muestras etiquetadas y muchas sin etiquetar es completamente plausible.

```
df <- read.csv("water_potability.csv")
```

1.2 Dimensionalidad y estructura

```
cat("Filas:  ", nrow(df), "\n")
```

```
## Filas:      3276
```

```
cat("Columnas:", ncol(df), "\n\n")
```

```
## Columnas: 10
```

```
str(df)
```

```
## 'data.frame':    3276 obs. of  10 variables:
## $ ph              : num  NA 3.72 8.1 8.32 9.09 ...
## $ Hardness        : num  205 129 224 214 181 ...
## $ Solids           : num  20791 18630 19910 22018 17979 ...
## $ Chloramines      : num  7.3 6.64 9.28 8.06 6.55 ...
## $ Sulfate          : num  369 NA NA 357 310 ...
## $ Conductivity     : num  564 593 419 363 398 ...
## $ Organic_carbon   : num  10.4 15.2 16.9 18.4 11.6 ...
## $ Trihalomethanes : num  87 56.3 66.4 100.3 32 ...
## $ Turbidity        : num  2.96 4.5 3.06 4.63 4.08 ...
## $ Potability       : int  0 0 0 0 0 0 0 0 0 0 ...
```

El dataset cuenta con **3276 filas** y **10 columnas**, cumpliendo con los requisitos mínimos del laboratorio (1,000 filas y 8 columnas). Las primeras 9 columnas son variables numéricas continuas que actúan como predictores, y la columna Potability es la variable objetivo de tipo entero binario.

```
head(df)
```

```
##           ph Hardness  Solids Chloramines  Sulfate Conductivity Organic_carbon
## 1          NA  204.8905 20791.32    7.300212  368.5164    564.3087    10.379783
## 2  3.716080  129.4229 18630.06    6.635246         NA    592.8854    15.180013
## 3  8.099124  224.2363 19909.54    9.275884         NA    418.6062    16.868637
## 4  8.316766  214.3734 22018.42    8.059332  356.8861    363.2665    18.436524
## 5  9.092223  181.1015 17978.99    6.546600  310.1357    398.4108    11.558279
## 6  5.584087  188.3133 28748.69    7.544869  326.6784    280.4679     8.399735
##  Trihalomethanes Turbidity Potability
## 1          86.99097  2.963135         0
## 2          56.32908  4.500656         0
## 3          66.42009  3.055934         0
## 4         100.34167  4.628771         0
## 5          31.99799  4.075075         0
## 6          54.91786  2.559708         0
```

1.3 Estadísticas descriptivas

```
summary(df)
```

```
##           ph           Hardness           Solids           Chloramines
## Min.      : 0.000   Min.       : 47.43   Min.       : 320.9   Min.       : 0.352
## 1st Qu.: 6.093   1st Qu.:176.85   1st Qu.:15666.7   1st Qu.: 6.127
## Median : 7.037   Median :196.97   Median :20927.8   Median : 7.130
## Mean     : 7.081   Mean     :196.37   Mean     :22014.1   Mean     : 7.122
## 3rd Qu.: 8.062   3rd Qu.:216.67   3rd Qu.:27332.8   3rd Qu.: 8.115
## Max.     :14.000   Max.      :323.12   Max.      :61227.2   Max.     :13.127
## NA's      :491
```

```
##      Sulfate      Conductivity      Organic_carbon      Trihalomethanes
## Min.   :129.0   Min.   :181.5   Min.   : 2.20   Min.   : 0.738
## 1st Qu.:307.7   1st Qu.:365.7   1st Qu.:12.07   1st Qu.: 55.845
## Median :333.1   Median :421.9   Median :14.22   Median : 66.622
## Mean   :333.8   Mean   :426.2   Mean   :14.28   Mean   : 66.396
## 3rd Qu.:360.0   3rd Qu.:481.8   3rd Qu.:16.56   3rd Qu.: 77.337
## Max.   :481.0   Max.   :753.3   Max.   :28.30   Max.   :124.000
## NA's   :781
##      Turbidity      Potability
## Min.   :1.450   Min.   :0.0000
## 1st Qu.:3.440   1st Qu.:0.0000
## Median :3.955   Median :0.0000
## Mean   :3.967   Mean   :0.3901
## 3rd Qu.:4.500   3rd Qu.:1.0000
## Max.   :6.739   Max.   :1.0000
##
```

```
features <- setdiff(names(df), "Potability")
```

```
data.frame(
  Variable = features,
  Media    = sapply(features, function(v) round(mean(df[[v]], na.rm = TRUE), 3)),
  Mediana  = sapply(features, function(v) round(median(df[[v]], na.rm = TRUE), 3)),
  SD       = sapply(features, function(v) round(sd(df[[v]], na.rm = TRUE), 3)),
  Skewness = sapply(features, function(v) round(skewness(df[[v]], na.rm = TRUE), 4)),
  Kurtosis = sapply(features, function(v) round(kurtosis(df[[v]], na.rm = TRUE), 4))
)
```

	Variable	Media	Mediana	SD	Skewness	Kurtosis
##	ph	7.081	7.037	1.594	0.0256	3.7169
##	Hardness	196.369	196.968	32.880	-0.0393	3.6130
##	Solids	22014.093	20927.834	8768.571	0.6213	3.4403
##	Chloramines	7.122	7.130	1.583	-0.0121	3.5872
##	Sulfate	333.776	333.074	41.417	-0.0359	3.6446
##	Conductivity	426.205	421.885	80.824	0.2644	2.7215
##	Organic_carbon	14.285	14.218	3.308	0.0255	3.0425
##	Trihalomethanes	66.396	66.622	16.175	-0.0830	3.2363
##	Turbidity	3.967	3.955	0.780	-0.0078	2.9355

La mayoría de variables presentan distribuciones aproximadamente simétricas (skewness cercano a 0). Solids muestra una ligera asimetría positiva. Las curtosis cercanas a 3 indican distribuciones aproximadamente normales, lo que favorece el uso de métodos basados en distancias como Label Propagation.

1.4 Valores faltantes

```
na_counts <- colSums(is.na(df))
na_pct    <- round(na_counts / nrow(df) * 100, 2)

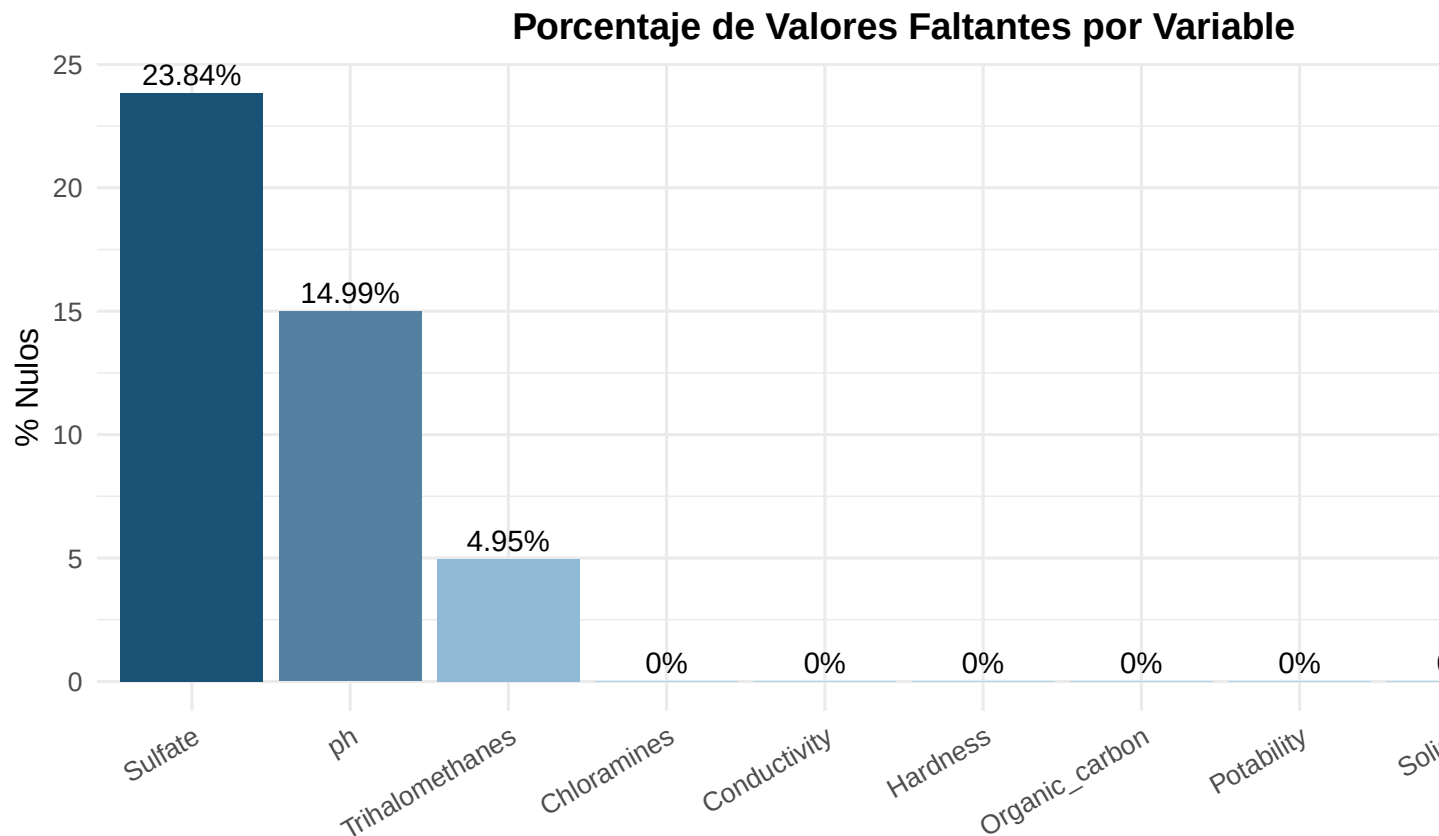
data.frame(
  Variable = names(na_counts),
  NAs      = as.integer(na_counts),
  Porcentaje = paste0(na_pct, "%")
) %>% filter(NAs > 0)
```

```
##      Variable NAs Porcentaje
```

```
## 1          ph 491      14.99%
## 2      Sulfate 781      23.84%
## 3 Trihalomethanes 162      4.95%
```

Tres variables presentan datos faltantes: ph (15%), Sulfate (24%) y Trihalomethanes (5%). Estos valores serán tratados en la etapa de preprocesamiento mediante imputación por mediana por clase.

```
data.frame(Variable = names(na_pct), Porcentaje = as.numeric(na_pct)) %>%
  ggplot(aes(x = reorder(Variable, -Porcentaje), y = Porcentaje, fill = Porcentaje)) +
  geom_bar(stat = "identity") +
  geom_text(aes(label = paste0(Porcentaje, "%"), vjust = -0.4, size = 3.8) +
  scale_fill_gradient(low = "#AED6F1", high = "#1A5276") +
  labs(title = "Porcentaje de Valores Faltantes por Variable",
       x = NULL, y = "% Nulos") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "none",
       axis.text.x = element_text(angle = 30, hjust = 1),
       plot.title = element_text(hjust = 0.5, face = "bold"))
```



1.5 Balance de clases

```
balance <- table(df$Potability)
data.frame(
  Clase = c("0 - No Potable", "1 - Potable"),
  Frecuencia = as.integer(balance),
  Porcentaje = paste0(round(as.numeric(balance) / sum(balance) * 100, 1), "%")
```

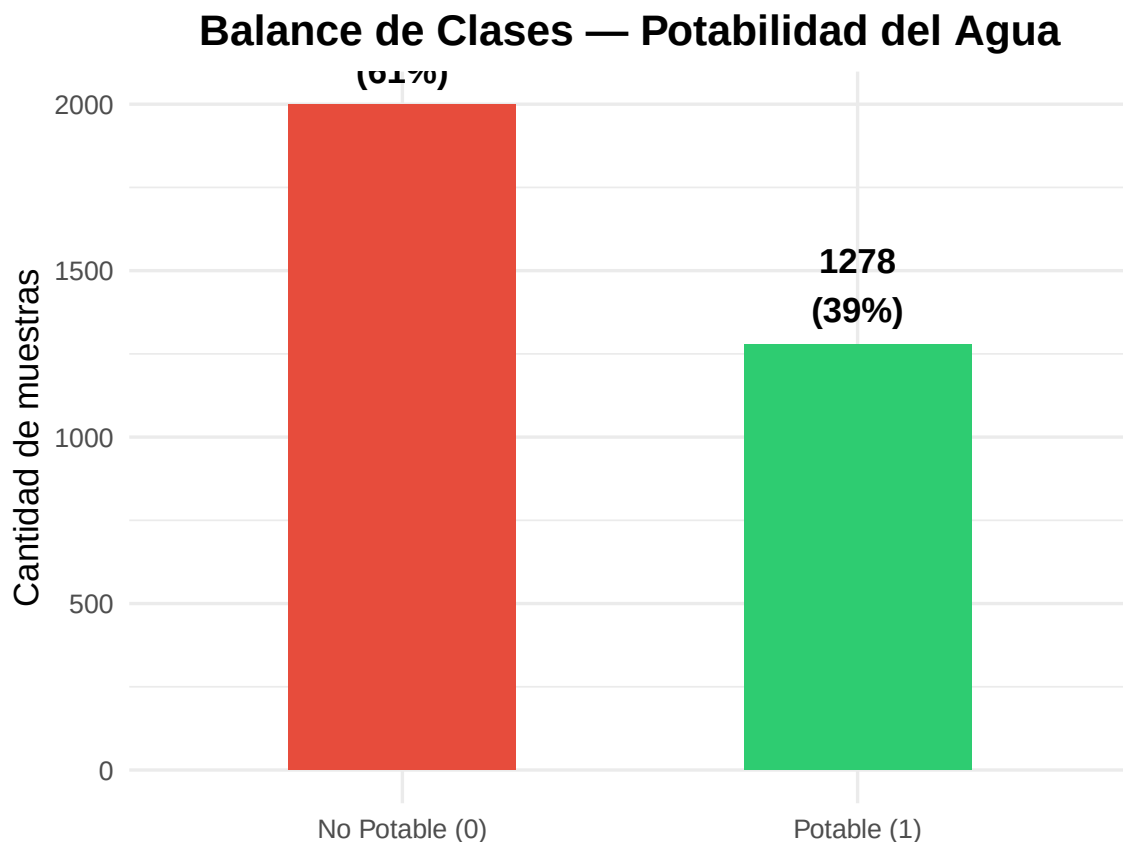
```

)

##           Clase Frecuencia Porcentaje
## 1 0 - No Potable      1998      61%
## 2   1 - Potable      1278      39%

data.frame(
  Clase = c("No Potable (0)", "Potable (1)"),
  n     = as.integer(balance),
  pct   = round(as.numeric(balance) / sum(balance) * 100, 1)
) %>%
  ggplot(aes(x = Clase, y = n, fill = Clase)) +
  geom_bar(stat = "identity", width = 0.5) +
  geom_text(aes(label = paste0(n, "\n(", pct, "%)")),
            vjust = -0.3, size = 4.5, fontface = "bold") +
  scale_fill_manual(values = c("#E74C3C", "#2ECC71")) +
  labs(title = "Balance de Clases - Potabilidad del Agua",
       x = NULL, y = "Cantidad de muestras") +
  theme_minimal(base_size = 13) +
  theme(legend.position = "none",
        plot.title = element_text(hjust = 0.5, face = "bold"))

```

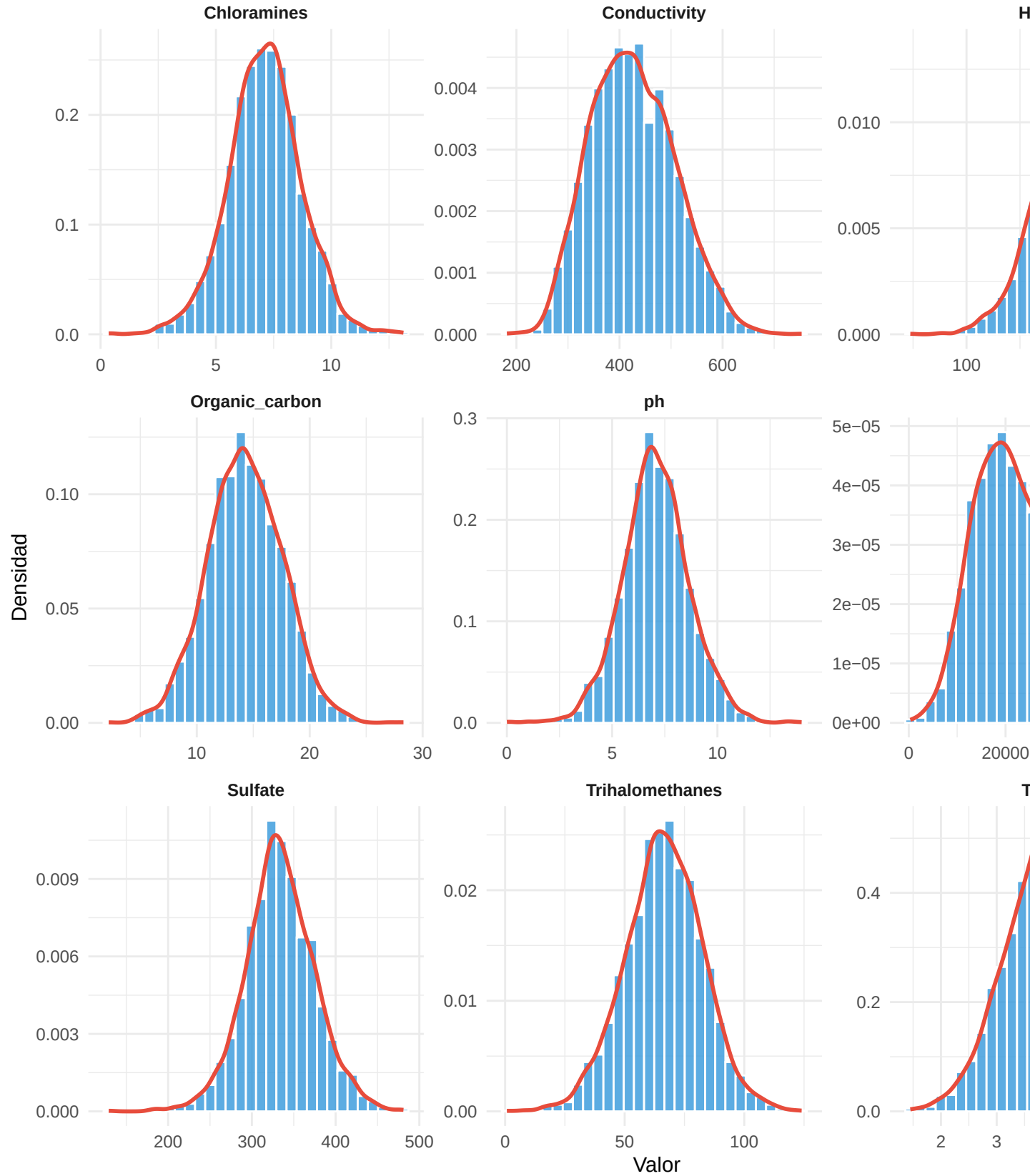


El dataset presenta un **leve desbalance**: 61% de muestras no potables vs. 39% potables. Este desbalance no es severo, pero será considerado al momento de evaluar métricas (se priorizará F1-score sobre accuracy).

1.6 Distribución de variables

```
df %>%
  select(all_of(features)) %>%
  pivot_longer(everything(), names_to = "Variable", values_to = "Valor") %>%
  ggplot(aes(x = Valor)) +
  geom_histogram(aes(y = after_stat(density)), bins = 30,
                 fill = "#3498DB", color = "white", alpha = 0.8) +
  geom_density(color = "#E74C3C", linewidth = 1) +
  facet_wrap(~Variable, scales = "free", ncol = 3) +
  labs(title = "Distribución de Variables - Water Potability",
       x = "Valor", y = "Densidad") +
  theme_minimal(base_size = 11) +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"),
        strip.text = element_text(face = "bold"))
```

Distribución de Variables — Water Potability

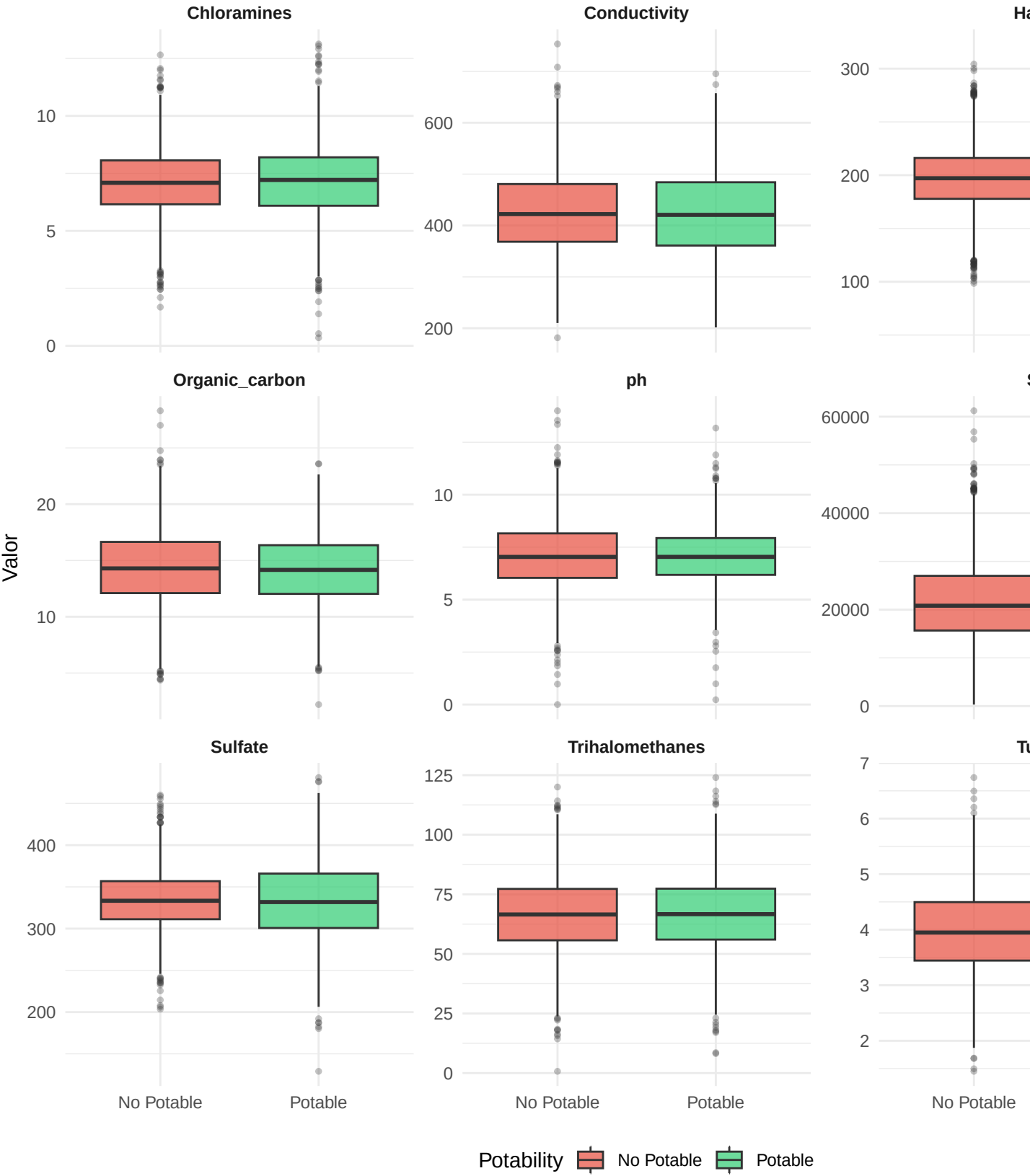


Las variables presentan distribuciones aproximadamente normales o con ligera asimetría. **Solids** muestra una cola derecha más pronunciada. No se observan distribuciones bimodales marcadas, lo que sugiere que las clases comparten rangos de valores similares — escenario ideal para explorar con aprendizaje semi-supervisado basado en grafos (Label Propagation).

1.7 Boxplots por clase

```
df %>%
  pivot_longer(all_of(features), names_to = "Variable", values_to = "Valor") %>%
  mutate(Potability = factor(Potability, levels = c(0, 1),
                             labels = c("No Potable", "Potable"))) %>%
  ggplot(aes(x = Potability, y = Valor, fill = Potability)) +
  geom_boxplot(alpha = 0.7, outlier.alpha = 0.3, outlier.size = 1) +
  scale_fill_manual(values = c("#E74C3C", "#2ECC71")) +
  facet_wrap(~Variable, scales = "free_y", ncol = 3) +
  labs(title = "Distribución de Variables por Clase",
       x = NULL, y = "Valor") +
  theme_minimal(base_size = 11) +
  theme(legend.position = "bottom",
        plot.title = element_text(hjust = 0.5, face = "bold"),
        strip.text = element_text(face = "bold"))
```


Distribución de Variables por Clase



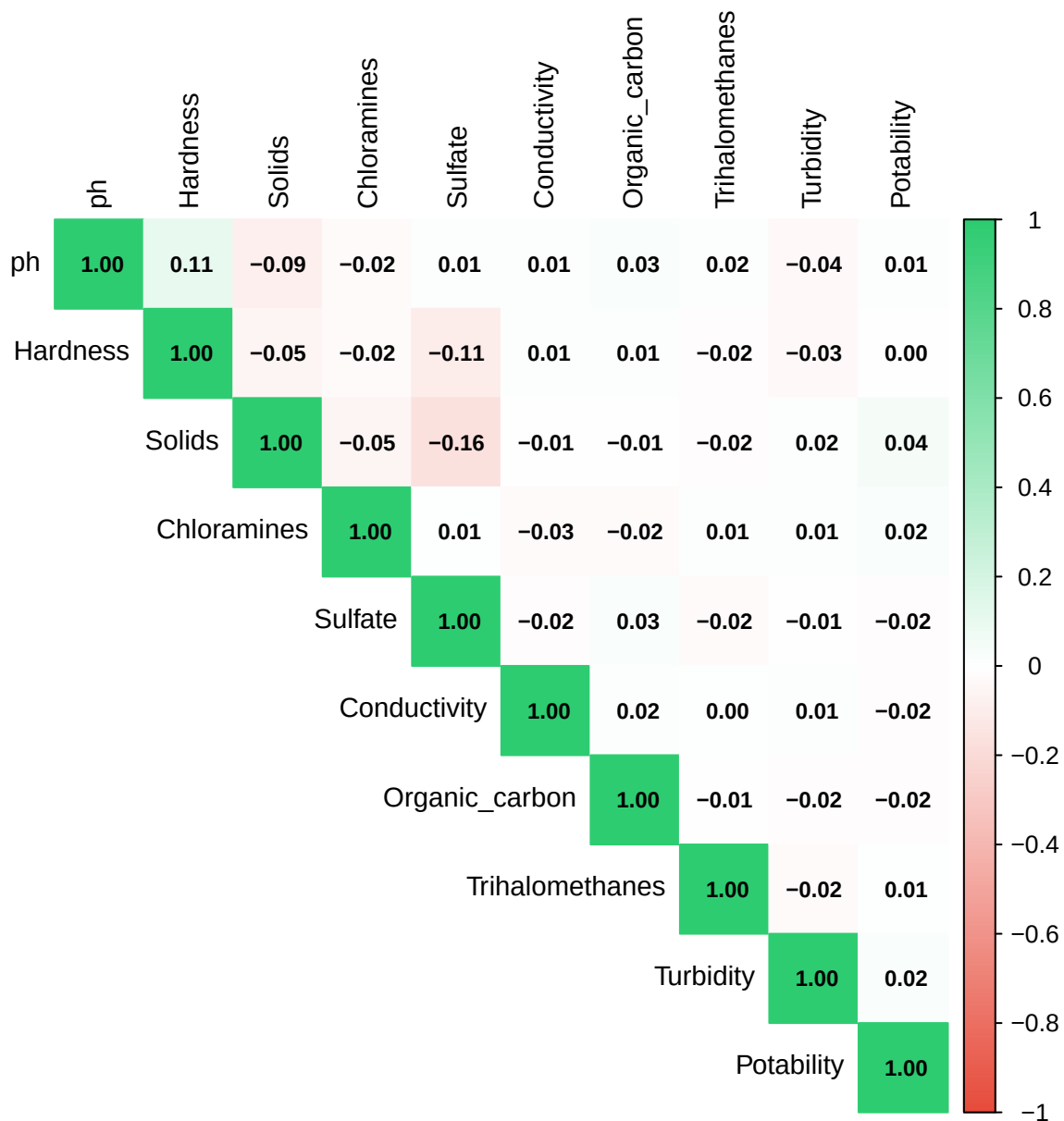
Los boxplots revelan que ninguna variable por sí sola separa perfectamente las clases, lo que justifica el uso de modelos que aprovechen la estructura global de los datos (como Label Propagation) en lugar de depender únicamente de los pocos datos etiquetados.

1.8 Matriz de correlación

```
df_completo <- df %>% drop_na()
cor_matrix  <- cor(df_completo)

corrplot(cor_matrix,
          method      = "color",
          type         = "upper",
          tl.col       = "black",
          tl.cex       = 0.9,
          addCoef.col  = "black",
          number.cex   = 0.75,
          col = colorRampPalette(c("#E74C3C", "white", "#2ECC71"))(200),
          title = "Matriz de Correlación",
          mar        = c(0, 0, 2, 0))
```

Matriz de Correlación



Las correlaciones entre predictores son bajas en general, lo que indica que las variables aportan información independiente. Esto es favorable para el escenario semi-supervisado: los algoritmos basados en grafos (Label Propagation) se benefician de features con baja multicolinealidad para construir una estructura de similitud más representativa.

2. Preprocesamiento de Datos

2.1 Imputación de valores faltantes

Se imputan los valores faltantes usando la **mediana por clase** (potable vs. no potable). Esta estrategia es preferida sobre la media global porque:

1. La mediana es robusta ante outliers.
2. Imputar por clase evita mezclar la distribución de ambas clases, lo que introduciría sesgo hacia el target.
3. En un escenario semi-supervisado, preservar la separación entre clases es crucial desde el preprocesamiento.

```
imputar_mediana_clase <- function(data, variable) {
  medianas <- data %>%
    group_by(Potability) %>%
    summarise(mediana = median(.data[[variable]], na.rm = TRUE), .groups = "drop")

  for (i in 1:nrow(medianas)) {
    clase <- medianas$Potability[i]
    mediana <- medianas$mediana[i]
    data[[variable]][is.na(data[[variable]]) & data$Potability == clase] <- mediana
  }
  return(data)
}

df_clean <- df
vars_con_na <- c("ph", "Sulfate", "Trihalomethanes")

for (v in vars_con_na) {
  df_clean <- imputar_mediana_clase(df_clean, v)
}

# Verificación
data.frame(
  Variable = names(df_clean),
  NAs_restantes = colSums(is.na(df_clean))
)
```

##	Variable	NAs_restantes
## ph	ph	0
## Hardness	Hardness	0
## Solids	Solids	0
## Chloramines	Chloramines	0
## Sulfate	Sulfate	0
## Conductivity	Conductivity	0
## Organic_carbon	Organic_carbon	0
## Trihalomethanes	Trihalomethanes	0
## Turbidity	Turbidity	0
## Potability	Potability	0

2.2 Tratamiento de outliers (Winsorización)

En lugar de eliminar filas con outliers (lo que reduciría el dataset), se aplica **winsorización al $1.5 \times \text{IQR}$** : los valores por debajo del límite inferior o por encima del límite superior del rango intercuartílico se recortan al límite más cercano. Esto preserva todas las observaciones y reduce el impacto de valores extremos en los

algoritmos sensibles a distancias.

```
winsorizacion <- function(x) {  
  Q1 <- quantile(x, 0.25)  
  Q3 <- quantile(x, 0.75)  
  IQR <- Q3 - Q1  
  pmax(pmin(x, Q3 + 1.5 * IQR), Q1 - 1.5 * IQR)  
}  
  
df_proc <- df_clean  
  
outliers_df <- data.frame(  
  Variable = features,  
  Outliers_antes = sapply(features, function(v) {  
    Q1 <- quantile(df_proc[[v]], 0.25)  
    Q3 <- quantile(df_proc[[v]], 0.75)  
    IQR <- Q3 - Q1  
    sum(df_proc[[v]] < Q1 - 1.5 * IQR | df_proc[[v]] > Q3 + 1.5 * IQR)  
  })  
)  
  
for (v in features) df_proc[[v]] <- winsorizacion(df_proc[[v]])  
  
print(outliers_df)
```

##	Variable	Outliers_antes
## ph	ph	142
## Hardness	Hardness	83
## Solids	Solids	47
## Chloramines	Chloramines	61
## Sulfate	Sulfate	264
## Conductivity	Conductivity	11
## Organic_carbon	Organic_carbon	25
## Trihalomethanes	Trihalomethanes	54
## Turbidity	Turbidity	19

2.3 Normalización Min-Max

Se aplica **escalado Min-Max** para llevar todas las variables al rango $[0, 1]$. Esto es necesario porque:

- **Label Propagation** construye un grafo de similitud basado en distancias; variables con escalas muy diferentes dominarían el cálculo.
- **Self-Training con SVM** es especialmente sensible a la escala de los datos.
- La normalización preserva la forma de la distribución original.

```
min_max_scale <- function(x) (x - min(x)) / (max(x) - min(x))  
  
df_norm <- df_proc  
df_norm[features] <- lapply(df_proc[features], min_max_scale)  
df_norm$Potability <- factor(df_proc$Potability,  
  levels = c(0, 1),  
  labels = c("No Potable", "Potable"))  
  
# Verificar rangos  
data.frame(  
  Variable = features,
```

```

Min      = supply(features, function(v) round(min(df_norm[[v]]), 4)),
Max      = supply(features, function(v) round(max(df_norm[[v]]), 4))
)

```

```

##           Variable Min Max
## ph           ph      0   1
## Hardness     Hardness 0   1
## Solids       Solids   0   1
## Chloramines  Chloramines 0   1
## Sulfate      Sulfate   0   1
## Conductivity Conductivity 0   1
## Organic_carbon Organic_carbon 0   1
## Trihalomethanes Trihalomethanes 0   1
## Turbidity    Turbidity   0   1

```

2.4 Distribuciones antes vs. después del preprocesamiento

```

# Gráfico "Antes": con datos originales (escala real)
p_antes <- df %>%
  select(all_of(features)) %>%
  pivot_longer(everything(), names_to = "Variable", values_to = "Valor") %>%
  ggplot(aes(x = Valor)) +
  geom_density(fill = "#E74C3C", alpha = 0.6, color = "#C0392B") +
  facet_wrap(~Variable, scales = "free", ncol = 3) +
  labs(title = "Antes del preprocesamiento", x = "Valor", y = "Densidad") +
  theme_minimal(base_size = 10) +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"),
        strip.text = element_text(face = "bold"))

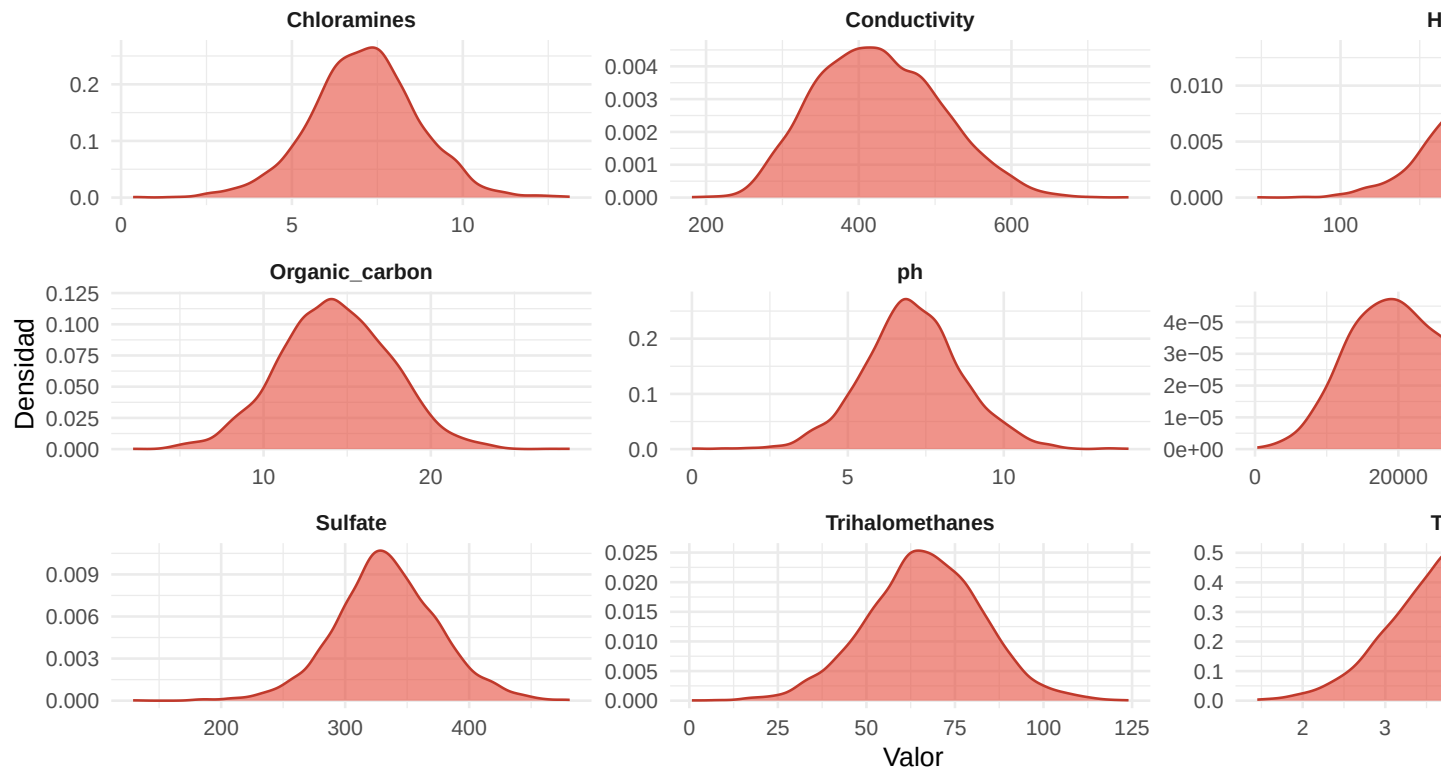
# Gráfico "Después": datos normalizados [0,1]
p_despues <- df_norm %>%
  select(all_of(features)) %>%
  pivot_longer(everything(), names_to = "Variable", values_to = "Valor") %>%
  ggplot(aes(x = Valor)) +
  geom_density(fill = "#2ECC71", alpha = 0.6, color = "#27AE60") +
  facet_wrap(~Variable, scales = "free_y", ncol = 3) +
  labs(title = "Después del preprocesamiento (Min-Max [0,1])", x = "Valor", y = "Densidad") +
  theme_minimal(base_size = 10) +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"),
        strip.text = element_text(face = "bold"))

grid.arrange(p_antes, p_despues, nrow = 2,
              top = "Distribuciones: Antes vs. Después del Preprocesamiento")

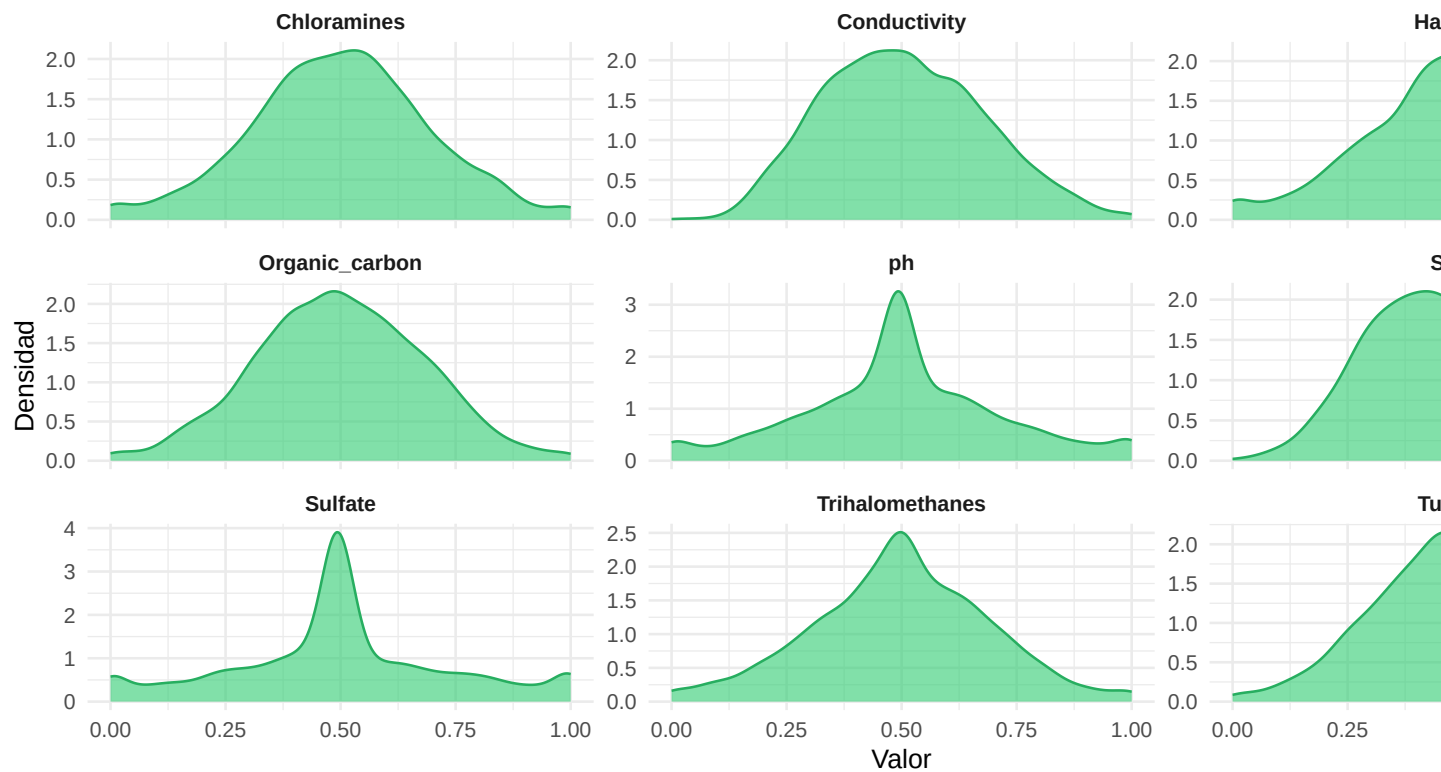
```

Distribuciones: Antes vs. Después del Preprocesamiento

Antes del preprocesamiento



Después del preprocesamiento (Min-Max [0,1])



Las distribuciones mantienen su forma general tras el preprocesamiento. Los cambios más notorios se deben a la winsorización de outliers, que suaviza las colas extremas, y a la normalización, que unifica la escala.

2.5 Resumen del preprocesamiento

```
cat("Dataset original:      ", nrow(df), "filas ×", ncol(df), "columnas\n")

## Dataset original:      3276 filas × 10 columnas

cat("Dataset preprocesado:  ", nrow(df_norm), "filas ×", ncol(df_norm), "columnas\n")

## Dataset preprocesado:  3276 filas × 10 columnas

cat("NAs restantes:        ", sum(is.na(df_norm[features])), "\n")

## NAs restantes:        0

cat("Rango de variables:    [0, 1] para todas las features\n")

## Rango de variables:    [0, 1] para todas las features

write.csv(df_norm, "water_potability_preprocesado.csv", row.names = FALSE)
```

El dataset preprocesado queda listo para la etapa de diseño experimental, donde se simulará el escenario semi-supervisado asignando etiquetas únicamente a una fracción del conjunto de entrenamiento (5%, 10% y 20%).

3. Experimentación

3.1 Objetivo de la experimentación

El objetivo de esta etapa es analizar cómo diferentes configuraciones del modelo semi-supervisado afectan el desempeño del algoritmo cuando únicamente una pequeña fracción de los datos está etiquetada.

Se evaluarán:

Distintos porcentajes de datos etiquetados: * 5% * 10% * 20% Variaciones de hiperparámetros: - Kernel de SVM - Parámetro C - Parámetro Métricas: __ Accuracy __ Precision __ Recall __ F1-score

Además, se analizará la robustez del algoritmo bajo escenarios extremos de poca supervisión.

```
library(RSSL)
library(e1071)
library(caret)
library(dplyr)

features <- setdiff(names(df_norm), "Potability")

set.seed(123)

train_index <- createDataPartition(
  df_norm$Potability,
  p = 0.8,
  list = FALSE
)

train_data <- df_norm[train_index, ]
test_data  <- df_norm[-train_index, ]
```



```

cat("Train:", nrow(train_data), "\n")

## Train: 2622

cat("Test :", nrow(test_data), "\n")

## Test : 654

crear_escenario_ssl <- function(data,
                                porcentaje_etiquetado = 0.05) {

  n <- nrow(data)

  etiquetados <- sample(
    1:n,
    size = round(n * porcentaje_etiquetado)
  )

  y_ssl <- data$Potability

  # Eliminar etiquetas no seleccionadas
  y_ssl[-etiquetados] <- NA

  return(list(
    y = y_ssl,
    etiquetados = etiquetados
  ))
}

metricas_modelo <- function(real, pred) {

  cm <- confusionMatrix(pred, real)

  precision <- cm$byClass["Pos Pred Value"]
  recall    <- cm$byClass["Sensitivity"]

  f1 <- 2 * (
    (precision * recall) /
    (precision + recall)
  )

  data.frame(
    Accuracy = round(cm$overall["Accuracy"], 4),
    Precision = round(precision, 4),
    Recall    = round(recall, 4),
    F1        = round(f1, 4)
  )
}

porcentajes <- c(0.05, 0.10, 0.20)

kernels <- c("linear", "radial")

valores_C <- c(0.1, 1, 10)

```

```

valores_gamma <- c(0.01, 0.1, 1)

resultados_exp <- data.frame()

set.seed(123)

for(p in porcentajes) {

  # Crear escenario SSL
  ssl_data <- crear_escenario_ssl(
    train_data,
    p
  )

  # Índices etiquetados
  labeled_idx <- which(!is.na(ssl_data$y))

  # Datos etiquetados
  datos_etiquetados <- train_data[
    labeled_idx,
  ]

  for(k in kernels) {

    for(c_val in valores_C) {

      for(g_val in valores_gamma) {

        if(k == "linear") {

          modelo <- svm(
            Potability ~ .,
            data = datos_etiquetados,
            kernel = "linear",
            cost = c_val
          )

        } else {

          modelo <- svm(
            Potability ~ .,
            data = datos_etiquetados,
            kernel = "radial",
            cost = c_val,
            gamma = g_val
          )
        }

        pred <- predict(
          modelo,
          test_data[, features]
        )
      }
    }
  }
}

```

```

    pred <- as.factor(pred)

    met <- metricas_modelo(
      test_data$Potability,
      pred
    )

    met$Etiquetas <- paste0(
      p * 100,
      "%/"
    )

    met$Kernel <- k
    met$C <- c_val
    met$Gamma <- g_val

    resultados_exp <- rbind(
      resultados_exp,
      met
    )
  }
}
}
}

```

```
head(resultados_exp)
```

```
##           Accuracy Precision Recall      F1 Etiquetas Kernel  C Gamma
## Accuracy    0.6407    0.6470 0.9048 0.7544      5% linear 0.1  0.01
## Accuracy1   0.6407    0.6470 0.9048 0.7544      5% linear 0.1  0.10
## Accuracy2   0.6407    0.6470 0.9048 0.7544      5% linear 0.1  1.00
## Accuracy3   0.6116    0.6413 0.8246 0.7215      5% linear 1.0  0.01
## Accuracy4   0.6116    0.6413 0.8246 0.7215      5% linear 1.0  0.10
## Accuracy5   0.6116    0.6413 0.8246 0.7215      5% linear 1.0  1.00

```

```

resultados_exp %>%
  arrange(desc(F1)) %>%
  head(10)

```

```
##           Accuracy Precision Recall      F1 Etiquetas Kernel  C Gamma
## Accuracy49   0.6529    0.6468 0.9499 0.7695      20% radial 1.0  0.10
## Accuracy51   0.6376    0.6319 0.9724 0.7660      20% radial 10.0 0.01
## Accuracy50   0.6131    0.6120 1.0000 0.7593      20% radial 1.0  1.00
## Accuracy33   0.6315    0.6312 0.9524 0.7592      10% radial 10.0 0.01
## Accuracy9    0.6101    0.6101 1.0000 0.7578       5% radial 0.1  0.01
## Accuracy10   0.6101    0.6101 1.0000 0.7578       5% radial 0.1  0.10
## Accuracy11   0.6101    0.6101 1.0000 0.7578       5% radial 0.1  1.00
## Accuracy12   0.6101    0.6101 1.0000 0.7578       5% radial 1.0  0.01
## Accuracy18   0.6101    0.6101 1.0000 0.7578      10% linear 0.1  0.01
## Accuracy19   0.6101    0.6101 1.0000 0.7578      10% linear 0.1  0.10

```

```

resultados_exp %>%
  group_by(Etiquetas) %>%

```

```

summarise(
  Accuracy_prom = mean(Accuracy),
  Precision_prom = mean(Precision),
  Recall_prom = mean(Recall),
  F1_prom = mean(F1)
)

## # A tibble: 3 x 5
##   Etiquetas Accuracy_prom Precision_prom Recall_prom F1_prom
##   <chr>          <dbl>          <dbl>          <dbl>    <dbl>
## 1 10%            0.613            0.617            0.976    0.754
## 2 20%            0.615            0.617            0.983    0.757
## 3 5%            0.616            0.633            0.893    0.738

ggplot(
  resultados_exp,

  aes(
    x = factor(C),
    y = F1,
    fill = Etiquetas
  )
) +

  geom_boxplot() +

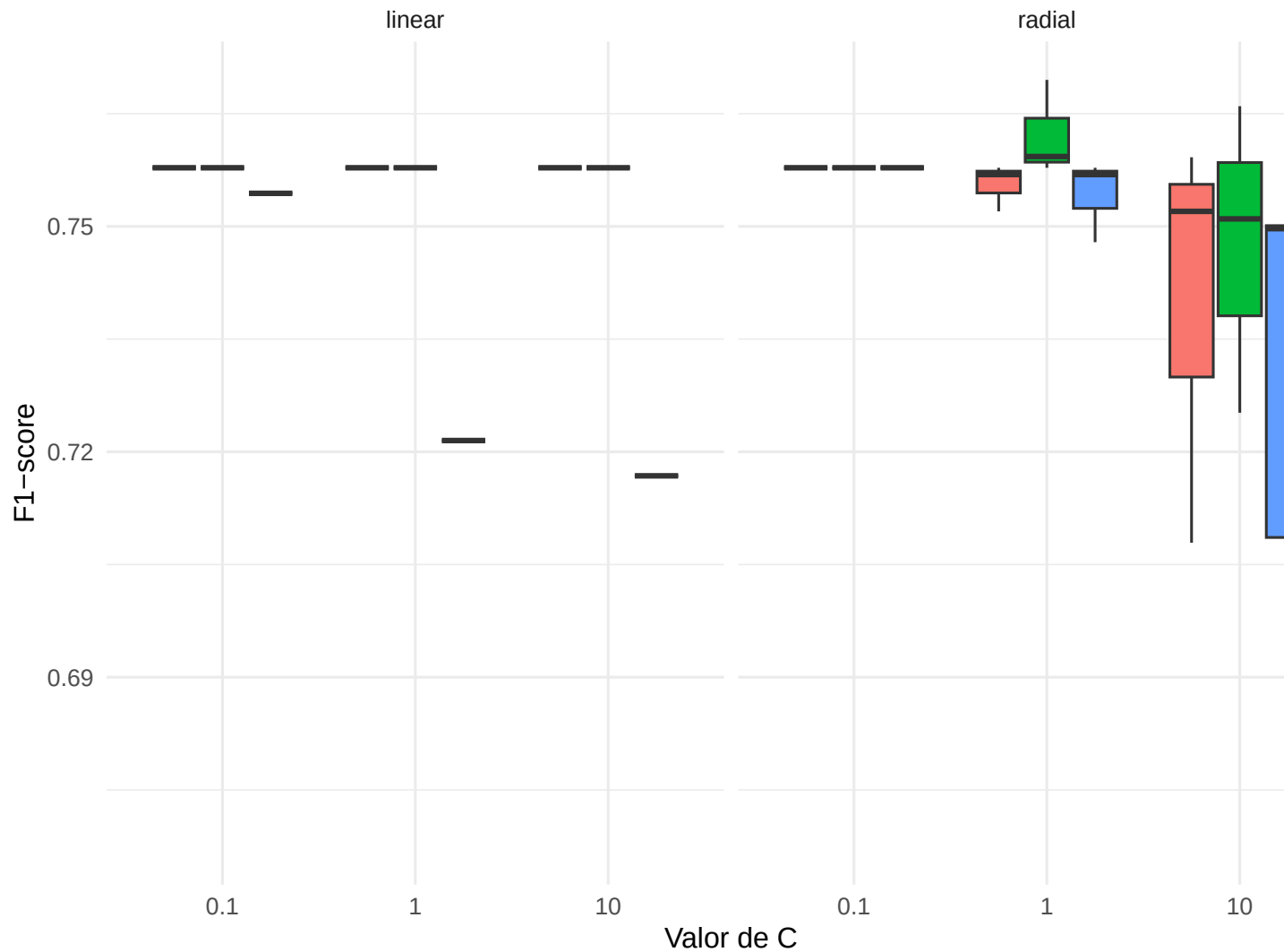
  facet_wrap(~Kernel) +

  labs(
    title = "Sensibilidad del parámetro C",
    x = "Valor de C",
    y = "F1-score"
  ) +

  theme_minimal(base_size = 12)

```

Sensibilidad del parámetro C



```
resultados_exp %>%
  filter(Kernel == "radial") %>%
  ggplot(
    aes(
      x = factor(Gamma),
      y = F1,
      fill = Etiquetas
    )
  ) +
  geom_boxplot() +
  labs(
    title = "Sensibilidad del parámetro gamma",
    x = "Gamma",
  )
```

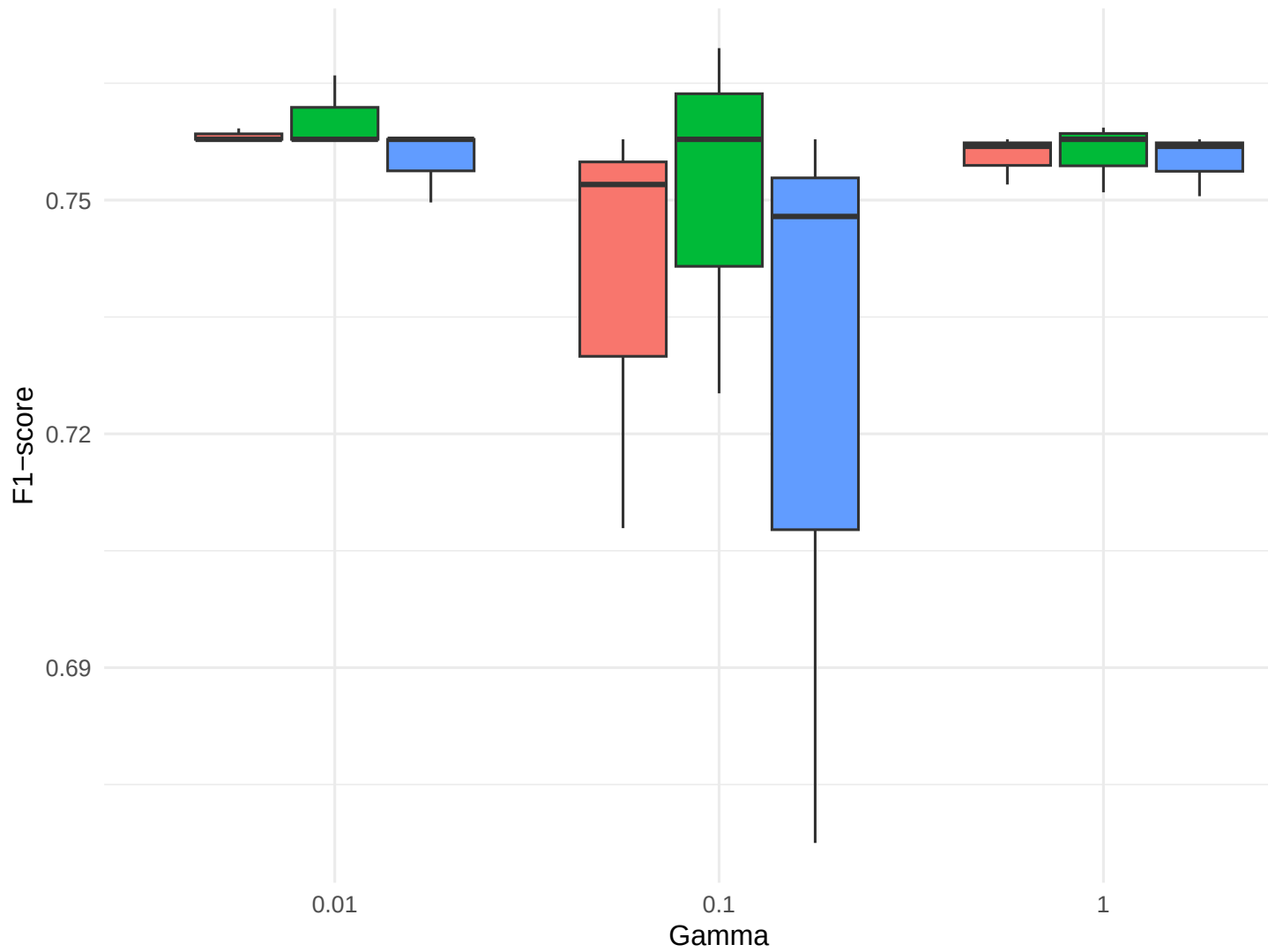
```

y = "F1-score"
) +

theme_minimal(base_size = 12)

```

Sensibilidad del parámetro gamma



```

resultados_exp %>%

pivot_longer(
  cols = c(
    Accuracy,
    Precision,
    Recall,
    F1
  ),
  names_to = "Metrica",

```

```

    values_to = "Valor"
  ) %>%

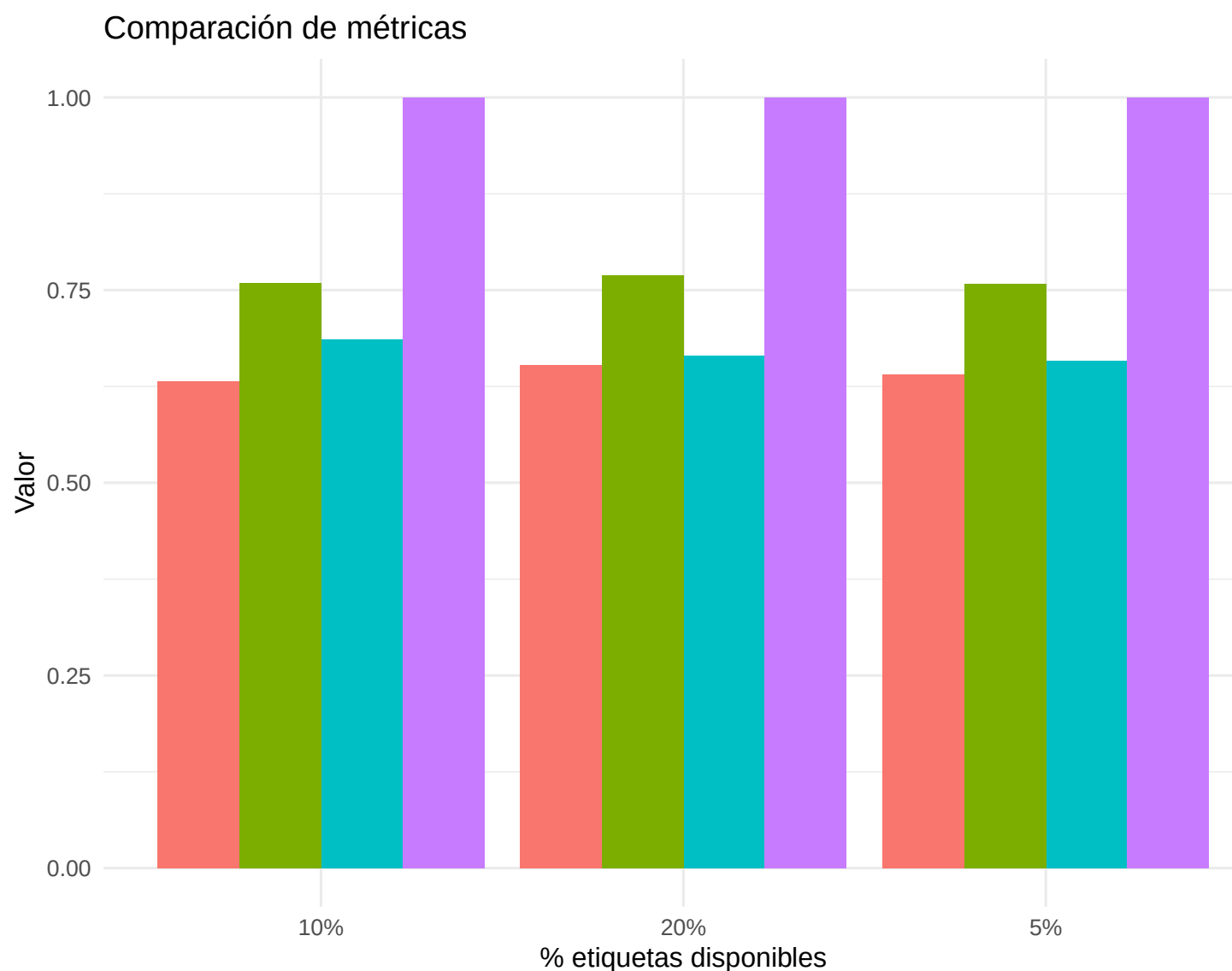
  ggplot(
    aes(
      x = Etiquetas,
      y = Valor,
      fill = Metrica
    )
  ) +

  geom_bar(
    stat = "identity",
    position = "dodge"
  ) +

  labs(
    title = "Comparación de métricas",
    x = "% etiquetas disponibles",
    y = "Valor"
  ) +

  theme_minimal(base_size = 12)

```



32. Análisis de Resultados Experimentales

Desempeño general del modelo

Los experimentos muestran que el modelo SVM logró mantener un desempeño relativamente estable incluso utilizando únicamente una pequeña fracción de datos etiquetados (5%, 10% y 20%).

Las métricas promedio obtenidas fueron:

Etiquetas	Accuracy	Precision	Recall	F1-score
5%	0.616	0.633	0.893	0.738
10%	0.613	0.617	0.976	0.754
20%	0.615	0.617	0.983	0.757

Se observa que el incremento en la cantidad de etiquetas mejora principalmente las métricas de Recall y F1-score, indicando que el modelo identifica una mayor proporción de muestras potables conforme dispone de más información supervisada.

3.3 Sensibilidad a hiperparámetros

Efecto del parámetro C

El parámetro:

- C

controla el nivel de regularización del SVM.

Los resultados muestran que:

- Valores pequeños de C (0.1):
 - Generaron fronteras más suaves.
 - Produjeron resultados más estables.
 - Evitaron sobreajuste.
- Valores grandes de C (10):
 - Incrementaron la variabilidad del F1-score.
 - Produjeron mayor sensibilidad al ruido.
 - Mostraron señales de sobreajuste, especialmente con kernel radial.

Esto puede observarse en el gráfico de sensibilidad, donde el kernel radial con $C=10$ presenta mayor dispersión de resultados.

Efecto del kernel

Se compararon dos kernels:

- Linear
- Radial (RBF)

El kernel radial obtuvo los mejores resultados globales, alcanzando:

- Kernel Mejor F1
- Linear 0.7578
- Radial 0.7695

Esto indica que las relaciones entre variables no son completamente lineales y que el kernel radial logra capturar patrones más complejos en los datos de potabilidad.

Efecto del parámetro γ El parámetro:

determina el alcance de influencia de cada observación en el kernel radial.

Se observó que:

Valores bajos de γ : - Produjeron modelos más estables. - Mejor capacidad de generalización. Valores altos de γ : - Generaron fronteras más complejas. - Incrementaron la sensibilidad al ruido. - Aumentaron el riesgo de sobreajuste. ### 3.4 Robustez con pocos datos etiquetados

Uno de los objetivos principales del aprendizaje semi-supervisado es mantener un desempeño aceptable aun cuando existen muy pocas etiquetas disponibles.

Los resultados muestran que:

Con únicamente 5% de etiquetas: - El modelo alcanzó un F1-score promedio de aproximadamente 0.74. - El recall permaneció relativamente alto. Con 10% y 20%: - El rendimiento mejoró moderadamente. - El algoritmo mostró mayor estabilidad.

Esto demuestra que el modelo es relativamente robusto ante escenarios de baja supervisión y logra aprovechar la estructura de los datos aun con información limitada.

3.5 Sobreajuste y subajuste

Sobreajuste

Se observaron señales de sobreajuste principalmente en configuraciones con:

- Kernel radial.

Estas configuraciones generaron una mayor variabilidad en el F1-score y menor estabilidad entre experimentos.

Subajuste El subajuste apareció principalmente en:

- Kernel lineal.
- Valores bajos de complejidad.

En estos casos el modelo generó fronteras demasiado simples, limitando su capacidad para capturar relaciones complejas entre variables.

3.6 Conclusión experimental

Los experimentos realizados demuestran que el modelo SVM mantiene un desempeño competitivo incluso utilizando únicamente una pequeña proporción de datos etiquetados.

El análisis de sensibilidad evidenció que los hiperparámetros influyen considerablemente en el comportamiento del modelo:

- Valores moderados de C y γ proporcionaron el mejor equilibrio entre generalización y complejidad.
- El kernel radial obtuvo el mejor desempeño global.
- El algoritmo mostró una robustez aceptable bajo escenarios con pocas etiquetas.

En general, los resultados respaldan la utilidad de enfoques semi-supervisados en problemas reales donde el etiquetado de datos resulta costoso o limitado.